

**Name:** Nilanjan Ghosh

**Register Number:** RA2311033010002

**Course:** Cognizant Digital Nurture 4.0

**Week:** 2

**Topic:** Spring Data JPA with Spring Boot, Hibernate

## **Hands-on 1: Spring Data JPA – Quick Example**

### **Aim**

To develop a Spring Boot application using Spring Data JPA and Hibernate to connect with a MySQL database and retrieve records from the country table.

---

### **Objective**

The objective of this exercise is to understand the implementation of Spring Data JPA by creating an entity, repository, and service layer, configuring the database connection, and retrieving data from MySQL using Hibernate.

---

### **Software Requirements**

- Java Development Kit (JDK)
  - Visual Studio Code
  - Apache Maven
  - MySQL Server 8
  - MySQL Workbench
  - Spring Boot
  - Spring Data JPA
  - Hibernate
- 

### **Theory**

Spring Data JPA is a framework that simplifies database operations by providing repository interfaces for performing CRUD operations without writing SQL queries manually.

Hibernate is the Object Relational Mapping (ORM) framework used by Spring Boot to map Java objects to database tables.

The application uses:

- **Entity** – Represents the database table.
  - **Repository** – Provides database access methods.
  - **Service** – Contains business logic.
  - **Spring Boot** – Manages application configuration and dependency injection.
  - **Hibernate** – Handles object-relational mapping.
- 

## Procedure

### Step 1: Create Spring Boot Project

A Spring Boot Maven project named **orm-learn** was created using Spring Initializr.

The following dependencies were included:

- Spring Boot DevTools
- Spring Data JPA
- MySQL Driver

**Screenshot 1:** Spring Initializr configuration.

---

### Step 2: Configure Database

A MySQL database named **ormlearn** was created.

A table named **country** was created and sample records were inserted.

SQL Commands:

```
CREATE DATABASE ormlearn;
```

```
USE ormlearn;
```

```
CREATE TABLE country(  
    code VARCHAR(2) PRIMARY KEY,  
    name VARCHAR(50)  
);
```

```
INSERT INTO country VALUES('IN','India');
```

```
INSERT INTO country VALUES('US','United States of America');
```

**Screenshot 2:** MySQL Workbench showing the database, table, and inserted records.

---

### Step 3: Configure Application Properties

The application.properties file was updated with:

- Database connection details
- Logging configuration
- Hibernate configuration

The application was configured to connect to the MySQL server running on **port 3307**.

**Screenshot 3:** application.properties file.

---

### Step 4: Create Entity Class

A persistence entity named **Country** was created.

Annotations used:

- @Entity
- @Table
- @Id
- @Column

These annotations map the Java class to the corresponding database table.

```
src > main > java > com > cognizant > ormlearn > model > Country.java > Country
10 public class Country {
11
12     @Id
13     @Column(name="code")
14     private String code;
15
16     @Column(name="name")
17     private String name;
18
19     public Country() {}
20
21     public String getCode() {
22         return code;
23     }
24
25     public void setCode(String code) {
26         this.code = code;
27     }
28
29     public String getName() {
30         return name;
31     }
32
33     public void setName(String name) {
34         this.name = name;
35     }
36
37     @Override
38     public String toString() {
39         return "Country [code=" + code + ", name=" + name + "]";
40     }
41 }
```

## Step 5: Create Repository

A repository interface named **CountryRepository** was created by extending JpaRepository.

This repository provides built-in CRUD operations.

```
src > main > java > com > cognizant > ormlearn > repository > CountryRepository.java > CountryRepository
1 package com.cognizant.ormlearn.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5
6 import com.cognizant.ormlearn.model.Country;
7
8 @Repository
9 public interface CountryRepository extends JpaRepository<Country,String>{
10
11 }
```

---

## Step 6: Create Service Layer

A service class named **CountryService** was created.

The repository was injected using `@Autowired`.

A method named `getAllCountries()` was implemented using:

`countryRepository.findAll();`

The method was annotated with `@Transactional`.

```
src > main > java > com > cognizant > ormlearn > service > CountryService.java > CountryService
1  package com.cognizant.ormlearn.service;
2
3  import java.util.List;
4
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.stereotype.Service;
7
8  import com.cognizant.ormlearn.model.Country;
9  import com.cognizant.ormlearn.repository.CountryRepository;
10
11 import jakarta.transaction.Transactional;
12
13 @Service
14 public class CountryService {
15
16     @Autowired
17     private CountryRepository repository;
18
19     @Transactional
20     public List<Country> getAllCountries(){
21
22         return repository.findAll();
23     }
24
25 }
26
```

---

## Step 7: Modify Main Class

The `OrmLearnApplication` class was modified to:

- Load the Spring Application Context.
- Obtain the `CountryService` bean.
- Invoke the `getAllCountries()` method

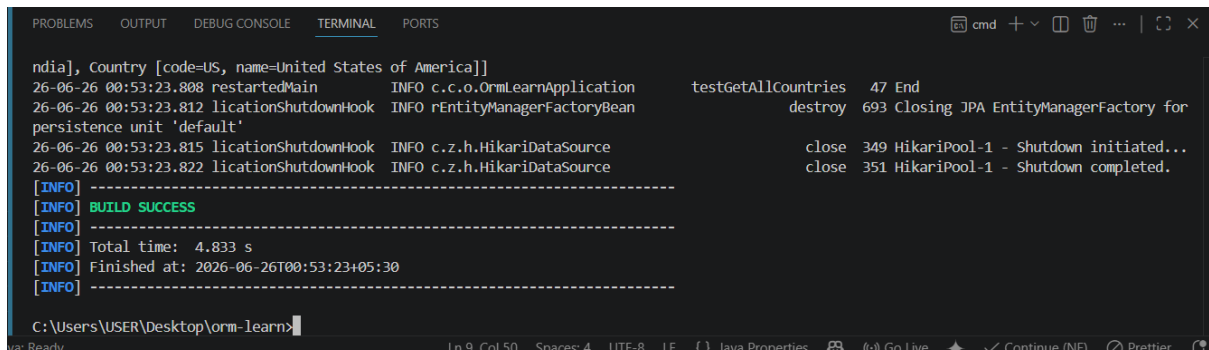
```
src > main > java > com > cognizant > ormlearn > OrmLearnApplication.java > OrmLearnApplication
1  package com.cognizant.ormlearn;
2
3  import java.util.List;
4
5  import org.slf4j.Logger;
6  import org.slf4j.LoggerFactory;
7
8  import org.springframework.boot.SpringApplication;
9  import org.springframework.boot.autoconfigure.SpringBootApplication;
10
11 import org.springframework.context.ApplicationContext;
12
13 import com.cognizant.ormlearn.model.Country;
14 import com.cognizant.ormlearn.service.CountryService;
15
```

---

## Step 8: Execute the Application

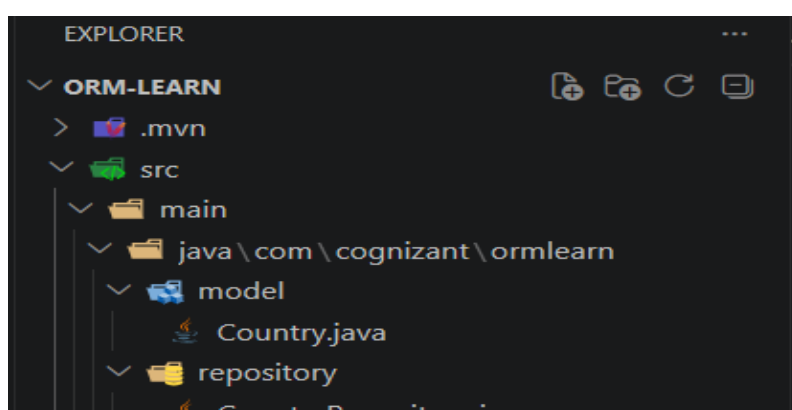
The application was executed successfully.

Hibernate established a connection with the MySQL database and retrieved all records from the **country** table.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ndia], Country [code=US, name=United States of America]]
26-06-26 00:53:23.808 restartedMain INFO c.c.o.OrmLearnApplication testGetAllCountries 47 End
26-06-26 00:53:23.812 licationShutdownHook INFO rEntityManagerFactoryBean destroy 693 Closing JPA EntityManagerFactory for
persistence unit 'default'
26-06-26 00:53:23.815 licationShutdownHook INFO c.z.h.HikariDataSource close 349 HikariPool-1 - Shutdown initiated...
26-06-26 00:53:23.822 licationShutdownHook INFO c.z.h.HikariDataSource close 351 HikariPool-1 - Shutdown completed.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.833 s
[INFO] Finished at: 2026-06-26T00:53:23+05:30
[INFO] -----
C:\Users\USER\Desktop\orm-learn>
```

## Project Structure



---

## Purpose of @SpringBootApplication

@SpringBootApplication is a convenience annotation that combines:

- @Configuration
- @EnableAutoConfiguration
- @ComponentScan

It enables automatic configuration, component scanning, and Java-based configuration, allowing the application to start with minimal setup.

---

## Dependency Hierarchy

The dependency hierarchy was verified using Maven.

Command:

mvn dependency:tree



The screenshot shows an IDE window with several tabs: application.properties, ormLearnApplication.java, Country.java (active), CountryRepository.java, and CountryService.java. The terminal output displays the Maven dependency tree for the project, listing various dependencies and their transitive dependencies.

```
src > main > java > com > cognizant > ormlearn > model > Country.java > Country
10 public class Country {
[INFO] +- org.springframework.boot:spring-boot-test-autoconfigure:jar:4.1.0:test
[INFO] +- com.jayway.jsonpath:json-path:jar:2.10.0:test
[INFO] +- jakarta.xml.bind:jakarta.xml.bind-api:jar:4.0.5:runtime
[INFO] | \- jakarta.activation:jakarta.activation-api:jar:2.1.4:runtime
[INFO] +- net.minidev:json-smart:jar:2.6.0:test
[INFO] | \- net.minidev:accessors-smart:jar:2.6.0:test
[INFO] | \- org.ow2.asm:asm:jar:9.7.1:test
[INFO] +- org.assertj:assertj-core:jar:3.27.7:test
```

---

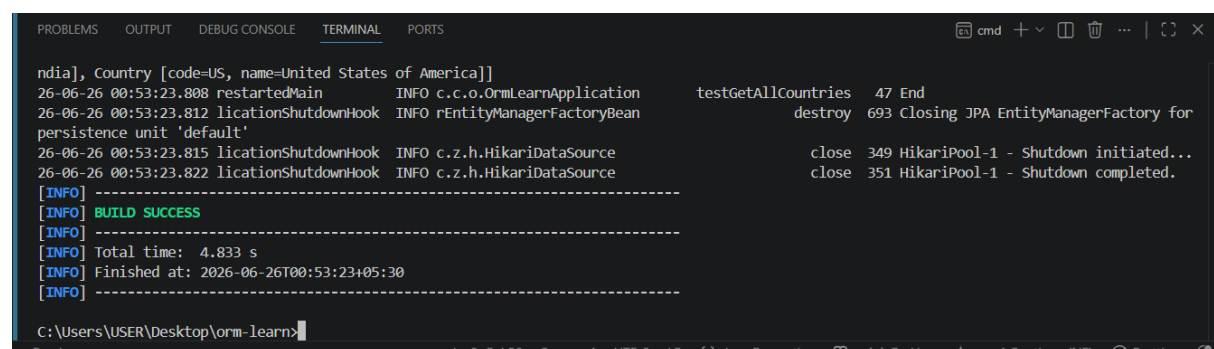
## Output

Inside main

Start

```
Countries = [Country(code=IN, name=India),  
              Country(code=US, name=United States of America)]
```

End



```
ndia], Country [code=US, name=United States of America]]
26-06-26 00:53:23.808 restartedMain      INFO c.c.o.OrmLearnApplication      testGetAllCountries  47 End
26-06-26 00:53:23.812 licationShutdownHook INFO rEntityManagerFactoryBean      destroy  693 Closing JPA EntityManagerFactory for
persistence unit 'default'
26-06-26 00:53:23.815 licationShutdownHook INFO c.z.h.HikariDataSource           close   349 HikariPool-1 - Shutdown initiated...
26-06-26 00:53:23.822 licationShutdownHook INFO c.z.h.HikariDataSource           close   351 HikariPool-1 - Shutdown completed.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time:  4.833 s
[INFO] Finished at: 2026-06-26T00:53:23+05:30
[INFO] -----
C:\Users\USER\Desktop\orm-learn>
```

---

## Result

The Spring Boot application successfully connected to the MySQL database using Spring Data JPA and Hibernate. The records from the **country** table were retrieved successfully through the repository and service layers.

---

## Conclusion

This exercise demonstrated the implementation of Spring Data JPA with Spring Boot and Hibernate. The application successfully established a connection with MySQL, mapped Java entities to database tables, and retrieved data using the repository pattern without writing SQL queries manually.

## Difference Between JPA, Hibernate, and Spring Data JPA

### Aim

To understand the differences between Java Persistence API (JPA), Hibernate, and Spring Data JPA and their roles in Java application development.

---

## Introduction

JPA, Hibernate, and Spring Data JPA are widely used technologies for database access in Java applications. Although they are closely related, each serves a different purpose.

- **JPA** is a specification.
  - **Hibernate** is an implementation of the JPA specification.
  - **Spring Data JPA** is a framework that simplifies data access by reducing boilerplate code and using JPA implementations such as Hibernate.
- 

## Java Persistence API (JPA)

Java Persistence API (JPA) is a Java specification (JSR 338) that defines a standard way to map Java objects to relational database tables.

### Features

- Standard specification for Object Relational Mapping (ORM).

- Defines annotations such as @Entity, @Table, @Id, and @Column.
- Does not provide any implementation.
- Requires an implementation such as Hibernate.

### **Advantages**

- Database independent.
  - Standard API for persistence.
  - Portable across different JPA providers.
- 

## **Hibernate**

Hibernate is an Object Relational Mapping (ORM) framework that implements the JPA specification.

### **Features**

- Implements all JPA interfaces.
- Converts Java objects into database tables.
- Automatically generates SQL queries.
- Supports caching, lazy loading, and transaction management.

### **Advantages**

- Reduces JDBC coding.
  - Supports automatic table mapping.
  - Provides high performance through caching.
- 

## **Spring Data JPA**

Spring Data JPA is a Spring Framework module that simplifies database programming by providing repository interfaces and automatic implementation of CRUD operations.

### **Features**

- Built on top of JPA.
- Uses Hibernate (or any other JPA provider) internally.
- Eliminates boilerplate DAO code.
- Provides built-in CRUD methods.

- Automatically manages transactions using Spring.

### **Advantages**

- Minimal coding.
  - Easy integration with Spring Boot.
  - Supports custom query methods.
  - Faster development.
- 

### **Hibernate Example**

Without Spring Data JPA, a Hibernate application requires manual session and transaction management.

```
public Integer addEmployee(Employee employee){
```

```
    Session session = factory.openSession();
```

```
    Transaction tx = null;
```

```
    Integer employeeID = null;
```

```
    try{
```

```
        tx = session.beginTransaction();
```

```
        employeeID = (Integer) session.save(employee);
```

```
        tx.commit();
```

```
    }
```

```
    catch(HibernateException e){
```

```
if(tx!=null)
```

```
tx.rollback();
```

---

### Spring Data JPA Example

Using Spring Data JPA, the same functionality can be implemented with very little code.

#### EmployeeRepository.java

```
public interface EmployeeRepository  
    extends JpaRepository<Employee,Integer>{  
  
}
```

#### EmployeeService.java

```
@Autowired  
private EmployeeRepository employeeRepository;  
  
@Transactional  
public void addEmployee(Employee employee){  
  
    employeeRepository.save(employee);  
  
}
```

---

### Comparison Table

| Feature | JPA           | Hibernate     | Spring Data JPA         |
|---------|---------------|---------------|-------------------------|
| Type    | Specification | ORM Framework | Spring Framework Module |

| Feature                | JPA                 | Hibernate                   | Spring Data JPA              |
|------------------------|---------------------|-----------------------------|------------------------------|
| Implementation         | No                  | Yes                         | No (uses JPA implementation) |
| ORM Support            | Defines ORM rules   | Provides ORM implementation | Uses Hibernate/JPA           |
| Boilerplate Code       | High                | Moderate                    | Very Low                     |
| Transaction Management | Not provided        | Manual                      | Automatic with Spring        |
| CRUD Operations        | Must be implemented | Manual coding               | Built-in repository methods  |
| Ease of Development    | Moderate            | Easy                        | Very Easy                    |

---

## Summary

- **JPA** defines the standard for persistence.
  - **Hibernate** implements the JPA specification and performs ORM operations.
  - **Spring Data JPA** further simplifies database programming by reducing boilerplate code and providing repository interfaces with automatic CRUD operations.
- 

## Conclusion

JPA, Hibernate, and Spring Data JPA work together to simplify database interaction in Java applications. JPA provides the standard specification, Hibernate implements the specification, and Spring Data JPA offers a higher level of abstraction that significantly reduces development effort by automatically handling repository implementation and transaction management.